

Project Handover Report — FH Saturation Dashboard

Generated: 2026-10-07 · **Written from the actual state of the project on disk** (nothing below is copied from a template). Everything marked **VERIFIED** was executed or queried while this report was written. Everything marked **NOT VERIFIED** was not.

Naming note. The request referred to a frontend folder called `frotnded enhancement`. No folder with that name exists. The real folders are listed below.

Part	Real folder (inside D:\lbnin\)
Backend (API)	dashboard_ericsson\
Frontend (web UI)	frontend-enhancement-for-dashboard\

A second, older copy of the frontend exists at `D:\samer\Documents\frontend-enhancement-for-dashboard`. It is **not** the integrated one. Ignore it.

---|---

| Backend (API) | dashboard_ericsson\ || Frontend (web UI) | frontend-enhancement-for-dashboard\ | (A second, older copy of the frontend exists at `D:\samer\Documents\frontend-enhancement-for-dashboard`. It is **not** the integrated one. Ignore it.)

Contents

1. [Quick start](#)
2. [Project overview](#)
3. [Folder structure](#)
4. [Technology stack](#)
5. [Prerequisites](#)
6. [Installation from zero](#)
7. [Environment variables](#)
8. [Database setup](#)
9. [Database structure \(models\)](#)
10. [Database indexes](#)
11. [Starting the application](#)
12. [Authentication](#)
13. [API reference](#)
14. [Import workflow](#)
15. [Calculation logic](#)
16. [Frontend architecture](#)
17. [Ericsson branding](#)
18. [No-measurement logic](#)

- 19. [Testing](#)
 - 20. [Troubleshooting](#)
 - 21. [Development workflow](#)
 - 22. [Production build](#)
 - 23. [Security](#)
 - 24. [Verified status and known issues](#)
 - 25. [Handover checklist](#)
-

1. QUICK START

Commands are for **Windows PowerShell**. Replace `<root>` with the folder that holds both projects.

```
# 1. Prerequisites: Node.js 22, pnpm 12 (or see §5), MongoDB 8 (Docker or local)

# 2. Terminal 1 – MongoDB (Docker route)
cd <root>\dashboard_ericsson
docker compose up -d

# 3. Terminal 2 – backend
cd <root>\dashboard_ericsson
npm install
copy .env.example config.env          # then EDIT config.env: set JWT_SECRET (>= 32 chars), ADMIN_EMAIL,
ADMIN_PASSWORD (>= 12 chars)
npm run create-admin                  # creates the first administrator; then delete ADMIN_PASSWORD from
config.env
npm run dev                           # API on http://localhost:3000

# 4. Terminal 3 – frontend
cd <root>\frontend-enhancement-for-dashboard
pnpm install
copy .env.example .env.local          # NEXT_PUBLIC_API_URL=http://localhost:3000/api/v1 (already the
default)
pnpm dev                              # UI on http://localhost:3001

# 5. Open http://localhost:3001 and log in with the admin you created.
#   Admin → "Import data" → choose congestion_data.xlsx.
```

Quick health check: `curl http://localhost:3000/api/v1/health` → `{"status":"success","message":"API is running",...,"database":"connected"}`.

The Docker route has **not** been run on the machine used to write this report (Docker is not installed there; see §24). The same steps with a non-Docker MongoDB were run and work.

2. PROJECT OVERVIEW

What the application does. It is a monitoring dashboard for microwave links (called *FH* in the code). An administrator uploads a daily Excel/CSV export of bandwidth-utilization histograms (20 slices per link per day, from 0–5 % up to 95–100 %). The system validates the file, stores it, computes a **load** per site, and shows how many sites are *Good*, *Medium*, *Critical* or have *No measurement*, plus the top/bottom sites, the top wilayas (Algerian provinces), and a detail page per site.

Frontend (Next.js, React): login screen, dashboard (counts, donut/bars, top 10 / bottom 10 sites, top 5 wilayas), site list with filters, site detail window, import dialog, import history and import detail (rows + issues). It contains **no business logic**: every number shown comes from the API.

Backend (Node.js, Express 5): JWT authentication, Excel/CSV import pipeline, calculation module, REST API, MongoDB persistence.

Communication



The browser calls the backend **directly** (no proxy). That is why the backend must allow the frontend's origin in `CORS_ORIGIN` (§7, §20).

Development vs production.

	Development	Production
Backend	<code>npm run dev</code> (nodemon restarts on change)	<code>npm start</code> (node server.js), <code>NODE_ENV=production</code>
Frontend	<code>pnpm dev</code> (next dev -p 3001)	<code>pnpm build</code> then <code>pnpm start</code> (next start -p 3001)
Build step	none	backend: none (plain JavaScript); frontend: <code>next build</code>

3. FOLDER STRUCTURE

3.1 Backend — dashboard_ericsson/

```
dashboard_ericsson/
├─ server.js          process entry: load config.env, check it, connect MongoDB, listen, graceful
shutdown
├─ app.js             the Express app (middleware + routes + error handler). No listen(): tests
import it
├─ package.json       scripts and dependencies
├─ package-lock.json  exact dependency versions (commit it)
├─ .env.example       template of the configuration (committed)
├─ config.env         REAL configuration, contains secrets – git-ignored, never commit
├─ docker-compose.yml local MongoDB 8
├─ jest.config.js     Jest configuration
├─ README.md          the backend's own README
├─ docs/openapi.yaml  the API reference (OpenAPI 3)
├─ dev-data/         git-ignored; holds congestion_data.xlsx (the real workbook used by some
tests)
├─ routes/           URL → controller, plus role checks
├─ controllers/      HTTP layer (read request, call a service, shape the answer) + the single
error handler
├─ models/           Mongoose schemas and indexes
├─ config/           thresholds.js, wilayas.js, reports.js (business configuration)
├─ scripts/createAdmin.js CLI to create/reset the administrator
├─ tests/            Jest tests (unit + real-database integration) and tests/helpers/
├─ utils/            everything else (see below)
├─   └─ calc/saturation.js PURE calculation module (no Express, no Mongoose)
├─   └─ excelReader.js    reads .xlsx/.csv into row objects
├─   └─ parsers.js        cell parsing (numbers, dates, text)
├─   └─ rowValidator.js   validates every row → accepted / rejected + issues
├─   └─ importPlanner.js  turns validated rows into sites / links / measurements to store
├─   └─ importService.js  DB-facing: whole import in one step, and revert
├─   └─ importLookup.js   find a successful import by id
├─   └─ dashboardService.js DB-facing: current state of every site
├─   └─ dashboardBuilder.js pure: states → dashboard blocks
├─   └─ siteList.js       pure: filters/sorts/shapes the site list
├─   └─ siteDetail.js     pure: records of one site → its page
├─   └─ wilaya.js, wilayaStats.js wilaya from NeId; per-wilaya aggregation
├─   └─ scopeParams.js, pagination.js validation of ?date=, ?import=, ?page=, ?limit=
├─   └─ security.js       helmet, CORS, rate limits, request id, request log
├─   └─ upload.js         multer (memory, 10 MB, .xlsx/.csv)
├─   └─ token.js, adminAccount.js, checkEnv.js, logger.js, appError.js, catchAsync.js
```

File / folder	Purpose	Modify?
<code>routes/*.js</code>	Which URL calls which controller; which roles	Yes, when adding an endpoint (also update <code>docs/openapi.yaml</code> : a test fails otherwise)
<code>controllers/*.js</code>	HTTP only. Keep logic out of here	Yes, thin changes
<code>utils/*.js</code> (services, builders)	The real logic	Yes, main place for backend changes
<code>utils/calc/saturation.js</code>	Business maths	Only with a documented business reason (\$15)
<code>models/*.js</code>	Schemas and indexes	Carefully: index changes affect existing data
<code>config/thresholds.js</code>	The 50 / 80 limits (provisional)	Yes, this is the one place for them
<code>config/wilayas.js</code> , <code>config/reports.js</code>	Wilaya names; texts of the site "Report" box	Rarely
<code>config.env</code>	Secrets and local settings	Locally yes, never commit
<code>package-lock.json</code> , <code>node_modules/</code>	Generated	Do not edit by hand
<code>dev-data/</code>	Local test workbook	Not in Git

3.2 Frontend — frontend-enhancement-for-dashboard/

```
frontend-enhancement-for-dashboard/
├─ app/
│   ├─ page.tsx           the ONLY page (route "/"): AuthProvider + Gate (login screen or dashboard)
│   ├─ layout.tsx         HTML shell, fonts, metadata
│   └─ ┬─ globals.css     Tailwind 4 theme + custom classes (e.g. .clean-input)
├─ components/
│   ├─ auth-provider.tsx  login / logout / session restore, React context
│   ├─ login-screen.tsx   login form (Ericsson logo)
│   ├─ network-dashboard.tsx the main screen: sidebar, header, Dashboard / Sites / Critical pages
│   ├─ site-details.tsx   "More info" window of one site
│   ├─ import-dialog.tsx  upload dialog (progress, errors)
│   ├─ history-page.tsx   import history + revert button + CSV export
│   ├─ import-detail.tsx  one import: summary, rows table, issues table
│   ├─ ui-bits.tsx        Loading, ErrorBox, Empty, Modal
│   └─ ┬─ ui/button.tsx   shadcn-style button
├─ lib/
│   ├─ api.ts             THE ONLY place that talks to the backend (token, errors, upload with
progress)
│   ├─ use-api.ts         useApi(path) hook for GET requests
│   ├─ types.ts           TypeScript types of the API responses
│   ├─ format.ts          date/percent formatting, condition labels/colors, noMeasurementText()
│   └─ ┬─ utils.ts        className helper
├─ public/ericsson-logo.png the Ericsson logo
├─ .env.example           NEXT_PUBLIC_API_URL
├─ next.config.mjs, tsconfig.json, postcss.config.mjs, components.json
├─ package.json, pnpm-lock.yaml, pnpm-workspace.yaml
├─ AGENTS.md, CLAUDE.md  notes written by Next.js tooling for AI agents; harmless
```

Normally modify	Do not modify by hand
components/, lib/, app/globals.css	.next/ (build output), node_modules/, pnpm-lock.yaml, next-env.d.ts, tsconfig.tsbuildinfo

4. TECHNOLOGY STACK

Versions are the **installed** versions read from node_modules / package.json.

Backend (plain JavaScript, CommonJS — **not TypeScript**)

Item	Version / note
Node.js	22.13.1 (the only tested version)
npm	10.9.2
Express	5.2.1

Item	Version / note
Mongoose	9.10.3
MongoDB server	8.0.4 tested (Docker image <code>mongo:8</code>)
JWT	<code>jsonwebtoken</code> 9.0.3 (HS256 only)
Password hashing	<code>bcryptjs</code> 3.0.3
Excel / CSV	<code>exceljs</code> 4.4.0
Upload	<code>multer</code> 2.4.0 (memory storage)
Security	<code>helmet</code> 8.3.0, <code>cors</code> 2.8.6, <code>express-rate-limit</code> 8.7.1
Config	<code>dotenv</code> 18.0.5
Tests	Jest 30.5.2 + Supertest 7.3.1 (not Vitest)
Dev server	<code>nodemon</code> 3.1.14
Validation library	none : validation is hand-written (<code>utils/rowValidator.js</code> , <code>utils/parsers.js</code> , <code>utils/checkEnv.js</code> , <code>utils/siteList.js</code> ...)
Logging library	none : own JSON logger <code>utils/logger.js</code>
TypeScript / linter / build	none (no typecheck, lint or build command exists for the backend)

Frontend

Item	Version / note
Framework	Next.js 16.3.3 (App Router), React 19.2.4
Language	TypeScript 5.7.3, <code>strict</code> on
CSS	Tailwind CSS 4.3.3 (<code>@tailwindcss/postcss</code>), <code>tw-animate-css</code> , <code>class-variance-authority</code> , <code>clsx</code> , <code>tailwind-merge</code>
UI	<code>@base-ui/react</code> 1.5.0, <code>shadcn</code> tooling, <code>lucide-react</code> 1.17.0 (icons)
Charts	no charting library : the donut and bars are drawn with CSS (<code>conic-gradient</code> via <code>conic()</code> in <code>lib/format.ts</code> , and <code>div</code> bars)
HTTP client	browser <code>fetch</code> and <code>XMLHttpRequest</code> (upload progress), wrapped in <code>lib/api.ts</code>
State management	React <code>useState</code> + one context (<code>AuthProvider</code>); no Redux/Zustand
Routing	single route / ; the "pages" (Dashboard / Sites / Critical / History) are React state, not URLs
Package manager	<code>pnpm@12.3.4</code> (declared in <code>package.json</code> , lockfile <code>pnpm-lock.yaml</code>)
Tests / linter	none configured

Next.js 16 differs from older versions. `AGENTS.md` asks to read `node_modules/next/dist/docs/` before relying on memory of older Next.js APIs.

5. PREREQUISITES

Tool	Version	Verify
Node.js	22.x (22.13.1 tested)	<code>node -v</code>
npm	10.x (ships with Node)	<code>npm -v</code>
pnpm	12.x (frontend)	<code>pnpm -v</code>
MongoDB	8.x , standalone (no replica set needed)	<code>mongod --version</code> or a Docker container
Docker Desktop	optional, only to run MongoDB	<code>docker -v</code> , <code>docker compose version</code>
Git	any recent	<code>git --version</code>
Free disk space	at least ~1 GB on the system drive , MongoDB refuses writes under 500 MB free	see §20

If `pnpm` cannot be installed, the frontend *might* install with `npm install`. This was **NOT VERIFIED** and would ignore `pnpm-lock.yaml` (exact versions then differ).

6. INSTALLATION FROM ZERO

```
# Get the code (the two projects are in one folder here; the repository layout may differ)
git clone <your-repository-url> <root>
cd <root>

# Backend
cd dashboard_ericsson
npm install
copy .env.example config.env      # see §7, then edit config.env

# Frontend
cd ..\frontend-enhancement-for-dashboard
pnpm install
copy .env.example .env.local
```

Notes:

- At the time of writing, the **backend** is a Git repository (with uncommitted changes, see §24); the **frontend** **has no `.git` folder**. The project root is not a repository either. Decide where the repository lives before sharing.
- The backend test workbook `dashboard_ericsson/dev-data/congestion_data.xlsx` is git-ignored: it is **not** in a fresh clone. Without it the tests that need it are reported as *skipped* (never as passed).

7. ENVIRONMENT VARIABLES

7.1 Backend — file `dashboard_ericsson/config.env`

The app reads `./config.env` (relative to the folder it is started from). It does **not** read `.env` or `.env.example`. Real environment variables also work and take precedence. The process refuses to start with a clear message if a required value is missing or invalid.

Variable	Purpose	Example	Required?	Secret?
<code>DATABASE_LOCAL</code>	MongoDB connection string used by the app	<code>mongodb://127.0.0.1:27017/fh-saturation</code>	Yes	If it contains credentials, yes
<code>JWT_SECRET</code>	Key that signs login tokens (≥ 32 characters)	<code><generate-a-random-secret></code>	Yes	YES
<code>JWT_EXPIRES_IN</code>	Token lifetime (<code>30m</code> , <code>12h</code> , <code>1d</code>)	<code>1d</code>	No (default <code>1d</code>)	No
<code>PORT</code>	HTTP port	<code>3000</code>	No (default <code>3000</code>)	No
<code>NODE_ENV</code>	<code>development</code> or <code>production</code>	<code>development</code>	No	No
<code>CORS_ORIGIN</code>	Browser origins allowed to call the API, comma-separated. Empty = no CORS headers (browsers block calls)	<code>http://localhost:3001</code>	No, but needed for the frontend	No
<code>RATE_LIMIT_MAX</code>	Requests per IP per 15 min, whole API	<code>300</code>	No (default <code>300</code>)	No
<code>LOGIN_RATE_MAX</code>	FAILED logins per IP per 15 min	<code>10</code>	No (default <code>10</code>)	No
<code>TRUST_PROXY</code>	Number of reverse proxies in front of the API	<code>1</code>	No (off)	No
<code>LOG_LEVEL</code>	<code>debug</code> , <code>info</code> , <code>warn</code> , <code>error</code> , <code>silent</code>	<code>info</code>	No (default <code>info</code>)	No
<code>DATABASE_TEST</code>	Base URL of test databases. Its name must contain "test" ; tests append a suffix and drop those databases	<code>mongodb://127.0.0.1:27017/fh-saturation-test</code>	Only for tests	No
<code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code>	Used only by <code>npm run create-admin</code> (password ≥ 12 chars, ≤ 72 bytes). Remove <code>ADMIN_PASSWORD</code> afterwards	<code><admin-email></code> , <code><admin-long-password></code>	Only for <code>create-admin</code>	<code>ADMIN_PASSWORD</code> : YES
<code>BCRYPT_COST</code>	Password hashing cost	<code>12</code>	No (default <code>12</code> ; tests lower it)	No

`dashboard_ericsson/.env.example` is the committed template. **Fix applied while writing this report:** its `CORS_ORIGIN` pointed to `http://localhost:5173`; it now says `http://localhost:3001`, which is where the frontend runs.

Generate a secret: `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`

7.2 Frontend — file `frontend-enhancement-for-dashboard/.env.local`

Variable	Purpose	Example	Required?
<code>NEXT_PUBLIC_API_URL</code>	Base URL of the backend API (including <code>/api/v1</code>)	<code>http://localhost:3000/api/v1</code>	No. The default in <code>lib/api.ts</code> is the same value

Important: `NEXT_PUBLIC_*` variables are **embedded into the JavaScript at build time**. After changing it, restart `pnpm dev`, or run `pnpm build` again for production. Everything with `NEXT_PUBLIC_` is visible to every user: **never put a secret in it**.

7.3 What must never be committed

`dashboard_ericsson/config.env` (git-ignored ✓), `.env.local` (git-ignored by `.env*.local` ✓), any real password, JWT secret or database URL with credentials. Rotate `JWT_SECRET` if it was ever shared.

8. DATABASE SETUP

Item	Value (from the code)
Database	MongoDB, standalone server (no replica set, no transactions used)
Default URL	<code>mongodb://127.0.0.1:27017/fh-saturation</code>
Database name	<code>fh-saturation</code>
Authentication	none in the default setup (the local container has no users). Add credentials to <code>DATABASE_LOCAL</code> for any shared server
Version	MongoDB 8 (image: <code>mongo:8</code> ; 8.0.4 tested)
Test databases	<code>DATABASE_TEST</code> + a suffix per test file (e.g. <code>fh-saturation-test-auth</code>), dropped by the tests

`docker-compose.yml` (as in the repository): service `mongo`, image `mongo:8`, container `fh-saturation-mongo`, port `127.0.0.1:27017:27017` (reachable from this machine only), named volume `mongo-data` mounted at `/data/db`, healthcheck with `mongosh`.

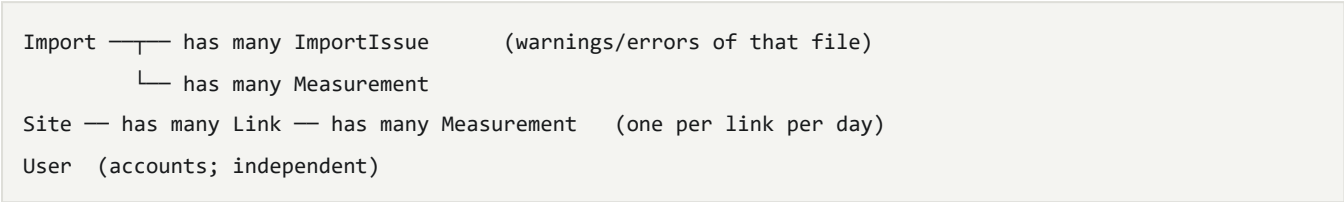
```
cd dashboard_ericsson
docker compose up -d      # start
docker compose ps         # check "healthy"
docker compose down       # stop (data is kept in the volume)
docker compose down -v    # stop AND DELETE the data
```

Without Docker: install MongoDB 8 and start it, for example `mongod --bind_ip 127.0.0.1 --port 27017 --dbpath <empty-folder>`, then keep `DATABASE_LOCAL` as is. (The development machine used `mongod-x64-win32-8.0.4.exe` from the `mongodb-memory-server` cache with `--dbpath D:\mongo-data`.)

Collections and indexes are created automatically by Mongoose when the app first connects (no migration or seed script exists besides `create-admin`).

9. DATABASE STRUCTURE (models)

Six models exist: **Site**, **Link**, **Import**, **ImportIssue**, **Measurement**, **User**. There is **no separate "Wilaya" collection**: wilayas are static reference data in `config/wilayas.js` (58 names), and a site's `wilayaCode` is derived from its Neld (`floor(neId / 1000)`; a Neld under 1000 has no wilaya).



Site (sites)

Field	Type	Notes
neId	Number (integer)	required
neType	String	required, trimmed
wilayaCode	Number ≥ 1, default null	derived from neId
createdAt , updatedAt	Date	timestamps

A site is identified by (**neId**, **neType**) (unique).

Link (links)

Field	Type	Notes
site	ObjectId → Site	required
measurePoint	String	required, trimmed
portRef , label , entityType	String, default null	
notInUse	Boolean, default false	
timestamps	Date	

Unique per (**site**, **measurePoint**).

Import (imports)

Field	Type	Notes
fileName , fileType (xlsx csv), fileSize		
sha256	String, 64 hex chars	unique : the same file cannot be imported twice
status	processing successful reverting	only <code>successful</code> ones are listed
counts	rowsTotal, valid, failed, rejected, skipped, withWarnings	

Field	Type	Notes
sitesCount, periodFrom, periodTo		
revertLinks, revertSites	ObjectId[], select:false	bookkeeping for an interrupted revert
createdAt	Date	shown as "imported on"

ImportIssue (importissues)

import (→ Import), sourceRow (Number), severity (error \| warning), code (String), message (String).

Measurement (measurements)

Field	Type	Notes
import, site, link	ObjectId	required
sourceRow	Number	row number in the file
endTime	Date	the day (midnight UTC)
status	OK PM_NOT_REACHABLE PM_INVALID PM_OTHER	
failure	String, default null	the failure text from the file
avgRaw, maxRaw, minRaw	Number	raw file values, unit unknown, never used in a calculation
bins	Number[20]	non-negative integers (seconds per 5 % slice)
totalSeconds, tailSeconds[20], meanUtil, p95		results; present only when status is OK (a non-OK row must not carry results; enforced by a validation hook)

condition (good/medium/critical) is **not stored**: it is computed when reading, from p95 and config/thresholds.js, so changing a threshold needs no data migration.

User (users)

email (unique, lowercase, ≤ 254), password (bcrypt hash, select:false, 12–72 bytes before hashing), role (admin \| viewer, default viewer), active (default true), passwordChangedAt, timestamps.

10. DATABASE INDEXES

VERIFIED against the live database fh-saturation (collection.indexes()):

Collection	Index	Unique	Why it exists
sites	neId_1_neType_1	yes	one site per (Neld, NeType)
sites	wilayaCode_1		wilaya counts and filters

Collection	Index	Unique	Why it exists
links	site_1_measurePoint_1	yes	one link per (site, measure point)
imports	sha256_1	yes	the same file can never be imported twice, even with two simultaneous uploads
imports	createdAt_-1		history, newest first
importissues	import_1_severity_1		issues of an import filtered by severity
importissues	import_1_sourceRow_1		issues in row order
measurements	link_1_endTime_1	yes	one measurement per link per day
measurements	endTime_1_status_1		the "latest day" state and condition counts
measurements	endTime_1_p95_-1		ranking by load
measurements	site_1_endTime_-1		site detail history
measurements	import_1_sourceRow_1		rows of an import in file order
users	email_1	yes	one account per email

(link, endTime) **unique index and duplicates**. The code does not use a field named `linkId`; the field is `link`. The index is `{ link: 1, endTime: 1 }`, unique. If a second measurement for the same link and day is inserted, MongoDB raises a duplicate-key error (code 11000). The import code avoids reaching that point: before storing, it looks up existing (link, day) pairs and **skips** matching rows, reporting them as a `DUPLICATE_OF_EXISTING` warning (the import's `counts.skipped`). If every row is already present the import fails with `409 IMPORT_NOTHING_NEW`. The unique index is the safety net that makes duplicates impossible even under concurrent requests.

11. STARTING THE APPLICATION

Use three terminals.

Terminal 1 — MongoDB (Docker route):

```
cd <root>\dashboard_ericsson
docker compose up -d
```

(or start your local `mongod`, §8)

Terminal 2 — backend:

```
cd <root>\dashboard_ericsson
npm run dev          # nodemon; or:  npm start
```

Expected log lines: `database connected`, then `server started` with `port: 3000`.

Terminal 3 — frontend:

```
cd <root>\frontend-enhancement-for-dashboard
pnpm dev          # next dev -p 3001
```

URL	What
http://localhost:3001	The application (login screen)
http://localhost:3000 /api/v1/health	Backend health check (no login needed)
(none)	There is no served API-documentation page . The API reference is the file <code>dashboard_ericsson/docs/openapi.yaml</code> ; open it in https://editor.swagger.io , Postman or Insomnia

12. AUTHENTICATION

- **Login:** `POST /api/v1/auth/login` with `{ "email", "password" }` returns `{ status, token, data: { user: { id, email, role } } }`. The token is a JWT signed with HS256 using `JWT_SECRET`; its lifetime is `JWT_EXPIRES_IN` (default `1d`).
- **Using the token:** every other endpoint (except `/health` and `/auth/login`) needs the header `Authorization: Bearer <token>`.
- **Checked on every request against the database:** a deleted or disabled account, or a password changed after the token was issued, invalidates the token at once; a role change applies at once. Only HS256 is accepted (a token claiming another algorithm is refused).
- **Roles:** `viewer` (read everything) and `admin` (read everything, plus `POST /imports` and `DELETE /imports/:id`). There is **no registration endpoint** on purpose. The first admin is created with `npm run create-admin`; other users are created by inserting a `User` document (`models/userModel.js`).
- **Errors:** `401` = missing / invalid / expired token (`TOKEN_MISSING`, `TOKEN_INVALID`, `TOKEN_EXPIRED`, `ACCOUNT_DISABLED`) or wrong password (`INVALID_CREDENTIALS`); `403` = wrong role.
- **Logout:** there is **no logout endpoint**. Tokens are stateless; the frontend logs out by deleting the token from `localStorage` (key `fh_token`). A token stays technically valid until it expires, unless the account is disabled or its password is changed.
- **Frontend behaviour (`lib/api.ts`, `components/auth-provider.tsx`):** the token is kept in `localStorage`. On page load the stored token is only trusted after `GET /auth/me` succeeds. Any `401` on a protected call logs the user out with the message "Your session has ended. Please log in again.". The import button is shown only to admins.

Example flow (no real credentials):

```

BASE=http://localhost:3000/api/v1
TOKEN=$(curl -s -X POST $BASE/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"<your-admin-email>","password":"<your-password>"}' \
  | node -pe "JSON.parse(require('fs').readFileSync(0,'utf8')).token")
curl -s $BASE/auth/me -H "Authorization: Bearer $TOKEN"

```

13. API REFERENCE

Base URL `http://localhost:3000/api/v1`. All responses are JSON. "Auth" = needs `Authorization: Bearer <token>`. Full detail, including examples, is in `docs/openapi.yaml` (a test fails if a route is added without being documented).

Common error shape (branch on `code`, not on `message`):

```

{ "status": "fail", "code": "IMPORT_DUPLICATE_FILE", "message": "...", "details": { }, "requestId": "..."}

```

`status` is `fail` for 4xx and `error` for 5xx. Stack traces and MongoDB messages are never sent. `requestId` also comes in the `X-Request-Id` header and matches the server log. Applies to every protected endpoint: `401` (token problems), `429` `RATE_LIMITED`, `503` `DATABASE_UNAVAILABLE`, `500` `INTERNAL_ERROR`.

Pagination (lists): `page` (default 1), `limit` (default and maximum differ per endpoint). Paged answers contain `results`, `total`, `page`, `pages`.

Health and authentication

Method	Path	Auth	Role	Purpose	Body / params	Response	Common errors
GET	<code>/health</code>	no	—	liveness + DB state	—	<code>{status, message, uptime, database}</code>	—
POST	<code>/auth/login</code>	no	—	log in	body <code>{email, password}</code>	<code>200 {status, token, data: {user}}</code>	<code>400</code> <code>VALIDATION_ERROR</code> , <code>401</code> <code>INVALID_CREDENTIALS</code> , <code>429</code> (10 failed attempts / 15 min / IP)
GET	<code>/auth/me</code>	yes	any	current user	—	<code>200 {data: {user}}</code>	<code>401</code>

Imports

Method	Path	Auth	Role	Purpose	Parameters / body	Response	Common errors
POST	/imports	yes	admin	upload + validate + store, in one request	multipart form, one file in field file (.xlsx / .csv , 10 MB max)	201 {message, data:{import:summary}}	400 UPLOAD_MISSING/UPLOAD_INVALID/UPLOAD_MALFORMED, 403, 409 IMPORT_DUPLICATE_FILE / IMPORT_NOTHING_NEW / IMPORT_IN_PROGRESS, 413 FILE_TOO_LARGE, 415 UNSUPPORTED_FILE_TYPE, 422 EXCEL_MISSING_COLUMNS / EXCEL_UNREADABLE / EXCEL_NO_SHEET / IMPORT_EMPTY_FILE / IMPORT_ALL_ROWS_REJECTED
GET	/imports	yes	any	history, newest first (successful only)	page, limit (default 20, max 100)	{results,total,page,pages, data:{imports:[{number,id,filename,fileType,importedAt,status,counts,sitesCount,periodFrom,periodTo}]}}	400
GET	/imports/:id	yes	any	summary + issue totals + days covered	—	{data:{import:{number,...summary, issues:{errors,warnings,byCode}, days:[{date,total,valid,failed}]}}	400, 404
GET	/imports/:id/rows	yes	any	the stored rows of an import	filter=all valid failed, q=<NeId digits>, page, limit (default 50, max 200)	{...,data:{rows:[{sourceRow,neId,neType,width,layerCode,measurePoint,...,endTime,status,failure,bins,totalSeconds,meanUtil,p95}]}}	400, 404

Method	Path	Auth	Role	Purpose	Parameters / body	Response	Common errors
GET	/imports/:id/issues	yes	any	warnings/errors of an import	severity=error warning, code=<UPPER_CASE>, page, limit (default 50, max 200)	{...,data:{issues:[{sourceRow,severity,code,message}]}}	400, 404
DELETE	/imports/:id	yes	admin	cancel an import and remove its data	—	200 {message:"Import reverted", data:{id,removed}}	400, 403, 404 (also on a 2nd call), 409 IMPORT_REVERT_IN_PROGRESS

There is **no** `POST /imports/:id/commit` and no preview step: see §14.

Dashboard

Method	Path	Auth	Role	Purpose	Parameters	Response	Errors
GET	/dashboard	yes	any	counts, rankings, wilayas	date=YYYY-MM-DD ("state as of that day"), import=<id>, limit (top/bottom size, default 10, max 50), wilayaLimit (default 5, max 58)	{data:{scope:{import,asOf,latestUpdate}, thresholds, totals:{sites,measured,noData}, byCondition:{critical,medium,good,no_data:{count,share}}, topSites[], bottomSites[], wilayas:{criticalSites, top[], unassigned}}}	400 bad date / limit, 404 unknown import

Sites

Method	Path	Auth	Role	Purpose	Parameters	Response	Errors
GET	/sites	yes	any	filtered, sorted, paged list	neId (digits; matches the start of the Neld), wilaya (code), condition=critical medium good no_data, sort=load_desc load_asc, date, import, page, limit (default 100, max 200)	{results,total,page,pages,from,to,data:{scope,thresholds,sites:[{siteId,neId,neType,wilayaCode,wilayaName,load,condition,noMeasurementReason,lastUpdate}]}}	400 invalid filter value

Method	Path	Auth	Role	Purpose	Parameters	Response	Errors
GET	/sites/:id	yes	any	one site's page (:id = siteId from the list)	date, import	{data: {scope, thresholds, site, current: {lastUpdate, load, condition, failureReasons, noMeasurementReason}, summary: {windowDays, measuredDays, average, maximum, minimum}, history[], links[] (each with bins[20]), report: {level, title, text}}}	400 bad id, 404 unknown site or no measurement in scope

Sort rule: sites **without** a measurement always come last.

Wilayas

Method	Path	Auth	Role	Purpose	Response
GET	/wilayas	yes	any	wilayas that contain at least one site	{results, data:{wilayas:[{code,name,sites}], unassignedSites}}

Wilaya statistics (critical counts, top 5) are part of /dashboard (data.wilayas). There is no separate statistics endpoint.

14. IMPORT WORKFLOW

Important difference from a "preview then commit" design: the backend does **upload** → **read** → **validate** → **plan** → **store in ONE request** (POST /imports). There is no preview endpoint and no commit endpoint. The "preview" is available **after** the import, through GET /imports/:id/rows and /issues, and the import can be undone with DELETE /imports/:id. The frontend mirrors this: the import dialog uploads, shows progress, and on success shows the result; the history page lets an admin open the import details or cancel it.

Lifecycle (utils/importService.js):

1. **Fingerprint.** SHA-256 of the file bytes. If an import with the same hash exists → 409 IMPORT_DUPLICATE_FILE (the answer says when it was imported). The unique index on sha256 also stops two simultaneous uploads (409 IMPORT_IN_PROGRESS).
2. **Create** an Import document with status processing.
3. **Parse** (excelReader.js): first sheet of the .xlsx, or the .csv (delimiter detected). Columns are found **by header name**. Required: NeId, NeType, EntityType, MeasurePoint, EndTime, Failure, Avg, Max, Min and the 20 slices 0-5 ... 95-100 (29 columns). Missing columns → 422 EXCEL_MISSING_COLUMNS (details.missing). Empty rows are ignored; no data rows → 422 IMPORT_EMPTY_FILE.
4. **Validate** every row (rowValidator.js). Outcome per row:
 - **accepted** — stored. Status OK (measured) or a PM failure (PM_NOT_REACHABLE, PM_INVALID, PM_OTHER) kept as coverage information. May carry **warnings**.
 - **rejected** — at least one error; not stored, only its issues are kept.

- A measured row's 20 slices must be integers ≥ 0 summing to **86 400 s**. A row with a `Failure` text is stored as a failure, never as a measurement.
- Issue codes seen in the code: errors `DUPLICATE_IN_FILE`, `INVALID_NEID`, `MISSING_NETYPE`, `MISSING_MEASUREPOINT`, `INVALID_ENDTIME`, `INVALID_NUMBER`, `BINS_*` (`BINS_LENGTH`, `BINS_NOT_INTEGER`, `BINS_NEGATIVE`, `BINS_ALL_ZERO`, `BINS_SUM_MISMATCH`); warnings `WILAYA_UNKNOWN`, `WILAYA_OUT_OF_RANGE`, `ENDTIME_NOT_MIDNIGHT`, `UNKNOWN_FAILURE`, `FAILURE_WITH_DATA`, `AVG_MAX_MIN_ORDER`, `NEID_MULTIPLE_NETYPE`, `DUPLICATE_OF_EXISTING`.

5. **Plan** (`importPlanner.js`): the distinct sites and links, and the measurements, to store. If every row was rejected → `422 IMPORT_ALL_ROWS_REJECTED`.
6. **Store** (`importService.js`): insert missing sites, then missing links, then skip rows whose (`link`, `day`) already exists (counted in `counts.skipped`, warning `DUPLICATE_OF_EXISTING`; all skipped → `409 IMPORT_NOTHING_NEW`), then insert measurements in chunks of 5 000, then the issues, then set status `successful` and the counts.
7. **Failure = nothing left behind**. Any error triggers a rollback: measurements, issues, and the sites/links created by this import (only if nothing else uses them) are deleted, and the `Import` document is removed. A failed import never appears in the history.
8. **Revert** (`DELETE /imports/:id`, admin): marks the import `reverting` (hidden), deletes its measurements and issues, then the links and sites nothing else uses. Safe to repeat (`404` the second time). If the server stops half-way, the import stays hidden and a new `DELETE` finishes the job after a 5-minute safety delay (`409 IMPORT_REVERT_IN_PROGRESS` before that).

Counts in an import: `rowsTotal`, `valid` (status OK), `failed` (PM failure rows, stored), `rejected`, `skipped`, `withWarnings`.

Reference dataset — the real workbook `dev-data/congestion_data.xlsx`. **VERIFIED** while writing this report:

Item	Expected	Verified
Rows	1,200	1,200 (read from the file)
Columns	29	29 (read from the file)
Valid / failed / rejected	862 / 338 / 0	862 / 338 / 0 (API, first import)
Rows with warnings	—	56 (= the 56 <code>importissues</code>)
Distinct sites / links	1,194 / 1,200	1,194 / 1,200 (database)
Days	10–15 Feb 2026	2026-02-10 ... 2026-02-15 (API)
Sites measured on 10/02	151	151 (<code>/dashboard?date=2026-02-10</code> → <code>measured: 151</code> , while a second import is also present)
Neld 18870, 10/02	2,192 s at ≥ 80 %	2,192 (<code>tailSeconds[16]</code>), total 86,400
S(80) of Neld 18870	2.54 %	2.54 % (2,192 / 86,400)

The current database holds a **second** import (`congestion_data_test.xlsx`, 1,200 rows, 868 valid, 332 failed, 250 sites, 2026-10-01 ... 2026-10-06), added by the project owner while testing. Dashboard totals therefore currently include both (1,324 sites). To see only one file use `?import=<id>`.

15. CALCULATION LOGIC

File: `dashboard_ericsson/utils/calc/saturation.js`. It is **pure**: no Express, no Mongoose, no I/O. Keep it that way: it is the single source of truth for the maths, and the frontend never recomputes it.

Concepts:

- A **bin** `i` (`0...19`) covers utilization `[5i, 5i+5)` percent. `bins[i]` = **seconds** the link spent in that slice during one day. Constants: `BIN_COUNT = 20`, `BIN_WIDTH = 5`, `EXPECTED_TOTAL_SECONDS = 86400`.
- `validateBins(bins)`: exactly 20 integers, none negative, not all zero, **sum = 86 400**. Otherwise returns a reason (`BINS_LENGTH`, `BINS_NOT_INTEGER`, `BINS_NEGATIVE`, `BINS_ALL_ZERO`, `BINS_SUM_MISMATCH`).
- `computeTails(bins) → tailSeconds[20]`: `tail[k]` = seconds at or above `5k` percent (a running sum from the top). `tail[0]` is the total (86 400). This array is **stored** on each measured document.
- `saturationRatio(tails, total, theta) → S(θ) = tail[θ/5] / total`, a share `0...1`. `θ` must be a multiple of 5 between 5 and 95 (otherwise `RangeError`). Example, Neld 18870: `tail[16] = 2192`, `S(80) = 2192 / 86400 = 2.54 %`.
- `meanUtilApprox(bins, total) → average utilization (%)` using slice mid-points: $\sum (5i + 2.5) \cdot bins[i] / total$. Stored as `meanUtil`.
- `percentileUtil(bins, total, p = 95) → the utilization level (%) exceeded only (100 - p) % of the time`, linear interpolation inside the slice, walking down from the top. With `p = 95`: the level exceeded 5 % of the day (about 72 minutes). Stored as `p95`. **This is the site's "load"**.
- `computeMetrics(bins) → one call for the import`: `{ ok, totalSeconds, tailSeconds, meanUtil, p95, s80 }` or `{ ok:false, reason }`.

How the load and condition are derived (outside `calc/`):

- **Site load** = the `p95` of the site's **worst measured link** on the site's **most recent day**. If every link failed that day, the load is `null` ("no measurement"): an unknown is never shown as an old value or as 0 % (`utils/dashboardService.js`).
- **Condition** (`utils/condition.js`, thresholds in `config/thresholds.js`): `load < 50 → good`; `50 ≤ load < 80 → medium`; `load ≥ 80 → critical`; `null → no_data`. Metric = `p95`.
- **The thresholds 50 and 80 are PROVISIONAL**. The file `config/thresholds.js` states they were copied from a mock-up and must be confirmed by a telecom engineer (open questions Q3, Q4). Change them in that one file only.

Things to know:

- `s80` is computed by `computeMetrics` but **is not stored and not returned by any endpoint**. There is **no theta request parameter** anywhere in the API. `S(θ)` can be derived from the stored `tailSeconds` if needed.
- The dashboard/site list "as of a day" uses the `date` parameter (measurements strictly before the end of that day).
- Site detail "last 7 days" = the last 7 days that have a record for the site (no gap filling).

16. FRONTEND ARCHITECTURE

- **Entry**: `app/page.tsx` renders `<AuthProvider><Gate/></AuthProvider>`. `Gate` shows `Loading...`, then `<LoginScreen/>` (anonymous) or `<NetworkDashboard/>` (authenticated).

- **Navigation:** a left sidebar (desktop) with **Dashboard, Sites, Critical**; a header with a Neld search box, a **History** button and (admins only) **Import data**. Pages are React state (`useState('Dashboard')`), not URLs: browser Back/Forward and deep links do not apply.
- **Dashboard page:** `useApi('/dashboard?date=&limit=10&wilayaLimit=5')`. Shows total / measured / critical metrics, the "Sites by status" donut, top-10 and bottom-10 cards, the top-5 wilayas, and a day filter ("state as of this day").
- **Sites / Critical pages:** `SitesPage` in `network-dashboard.tsx`. Filters: Neld (debounced 300 ms), wilaya (from `/wilayas`), day, condition (locked to `critical` on the Critical page), sort direction. Pagination 50 per page (`PAGE_SIZE = 50`); the page resets to 1 when a filter changes.
- **More info:** `components/site-details.tsx` opens a modal with `GET /sites/:id?date=`: identity, condition, load summary (average/max/min), the 7-day bars, a link selector, the **20-slice histogram** of the selected link, and the report text.
- **Import:** `import-dialog.tsx` uploads with progress (`uploadImport()` in `lib/api.ts`); after success or revert, a `refreshKey` reloads dashboard and wilayas. `history-page.tsx` lists imports, opens `import-detail.tsx` (rows with filter all/valid/failed and Neld search, issues with severity filter), offers revert (admin) and a CSV export of the history.
- **API layer:** all HTTP goes through `lib/api.ts` (`api()`, `qs()`, `uploadImport()`, `tokenStore`, `ApiError`). The base URL is `API_URL = NEXT_PUBLIC_API_URL` (default `http://localhost:3000/api/v1`). `lib/use-api.ts` is the GET hook (loading / error / data / reload; ignores answers that arrive too late). Types of every response are in `lib/types.ts`.
- **Error handling:** `friendly()` in `lib/api.ts` turns status codes into user messages (401 session ended, 403 no permission, 413 file too large, 429 "wait N seconds", 503 unavailable, other 5xx generic, network error "Cannot reach the server"). Raw server errors are never shown.
- **Charts:** CSS only (no library), see §4.
- **Auth:** see §12.

17. ERICSSON BRANDING

- **Asset:** `frontend-enhancement-for-dashboard/public/ericsson-logo.png` — the wide Ericsson logo (wordmark + symbol), cropped tight, black on a **transparent** background, 2578 × 528 px (so it stays sharp at any size). Served by Next.js at `/ericsson-logo.png`.
- **Where it is used:**
 - Login page — `components/login-screen.tsx`, centered at the top of the card, 200 px wide.
 - Home/dashboard — `components/network-dashboard.tsx`, top of the left sidebar, 132 px wide.
 - Site details — the modal does **not** show a logo, so none was added.
- **How it is referenced:** a plain ``; width is set in CSS and the height follows the aspect ratio.
- To replace the logo: overwrite the PNG (keep a transparent background and the same file name) or change the `src` in the two components.

18. NO-MEASUREMENT LOGIC

A site has **no measurement** when every link of its most recent day failed (status `PM_NOT_REACHABLE`, `PM_INVALID` or `PM_OTHER`). Its `load` is `null` and its `condition` is `no_data`.

- **Where the reason comes from:** the backend stores the failure text of every non-OK row in `measurements.failure` (in the reference data: PM Failure - NE not reachable and PM Failure - NE PM data is invalid). The backend exposes the reason as `noMeasurementReason`:
 - `GET /sites` → each site: the distinct failure texts of its latest day, joined with ";" (or the status code if a row has no text); `null` when the site has a measurement. Built in `utils/dashboardService.js` (`computeSiteStates`) and returned by `utils/siteList.js` (`toListItem`).
 - `GET /sites/:id` → `data.current.noMeasurementReason` (same rule, from `utils/siteDetail.js`). `current.failureReasons` (the list form) is still returned.
- **Sites list — Load column:** a measured site shows the bar and the percentage. A site without a measurement shows **"No measurement"** and, beneath it, the reason in red (truncated with the full text as a tooltip).
- **More info:** for a site without a measurement the Load summary box shows **Measurement: No measurement** and **Reason: ...**. Per link, the message "No measurement for this link: ..." uses the same helper. A site with a measurement is unchanged.
- **One helper:** `noMeasurementText(reason)` in `lib/format.ts` is the single place that formats the reason for every screen. If the backend returns no reason it shows "No reason reported" (nothing is invented). Reasons are **not** hard-coded in the frontend.
- **Ranking:** sites with no measurement are counted but never ranked, and always listed last.

19. TESTING

Backend (run from `dashboard_ericsson/`; MongoDB must be running for the database tests):

Command	What
<code>npm test</code>	the whole Jest suite (26 suites, 428 tests)
<code>npx jest --runInBand</code>	the whole suite, one file at a time (recommended on weak/limited machines , see below)
<code>npx jest security</code>	one file by name
(none)	there is no typecheck, lint or build command for the backend

- Unit tests need nothing. The `.api.test.js`, `.db.test.js`, `security`, `import.*` and `apiDocs` tests use a real MongoDB, each in its own database (`DATABASE_TEST` + suffix, dropped at the end). `DATABASE_TEST` must contain "test".
- Tests that need the real workbook look for `dev-data/congestion_data.xlsx`; without it they are *skipped*.
- `tests/apiDocs.test.js` fails if a route is not documented in `docs/openapi.yaml`, or if the document names an error code the code never sends.
- Important suites: `sites.api`, `siteList`, `siteDetail` (list/detail and the no-measurement reason), `dashboard.api`, `dashboardBuilder`, `import.api`, `import.history`, `import.revert`, `importPlanner`, `rowValidator`, `excelReader`, `parsers`, `saturation` (calc module), `wilayas`, `wilayas.api`, `auth.`, `security`, `errorHandler`, `models.`, `userModel.db`.
- **Result VERIFIED on 2026-10-07:** `npx jest --silent --runInBand` → **Test Suites: 26 passed, 26 total · Tests: 428 passed, 428 total** (none skipped: the workbook is present).
- **Warning observed:** running the suite **in parallel** (`npm test` default) crashed the local `mongod` twice on the development machine ("out of memory") and produced 51 and 207 spurious failures. Serial runs were stable. Likely related to the full system drive (§24), but the cause was not confirmed.

Frontend (run from `frontend-enhancement-for-dashboard/`):

Command	What
<code>pnpm typecheck</code>	<code>tsc --noEmit</code>
<code>pnpm build</code>	<code>next build</code> (also runs the TypeScript check)
(none)	no lint script and no test runner is configured

20. TROUBLESHOOTING

Problem	Likely cause	Solution
MongoDB won't start / crashes with "out of memory"	system drive full; too many connections (parallel tests); old binary	free disk space (MongoDB needs > 500 MB free); run tests with <code>--runInBand</code> ; try <code>--wiredTigerCacheSizeGB 1</code> ; prefer the Docker image <code>mongo:8</code>
could not connect to the database, process exits	MongoDB not running, wrong <code>DATABASE_LOCAL</code>	start MongoDB (<code>docker compose up -d</code>), check the URL and port 27017
503 <code>DATABASE_UNAVAILABLE</code>	API up but MongoDB went away	restart MongoDB; the API recovers by itself
Port already in use (3000 / 3001 / 27017)	another process	<code>Get-NetTCPConnection -LocalPort 3000 → Stop-Process -Id <pid></code> ; or change <code>PORT</code> (backend) / the <code>-p</code> in the frontend scripts and update <code>CORS_ORIGIN</code> + <code>NEXT_PUBLIC_API_URL</code>
Invalid configuration in <code>config.env</code> : <code>JWT_SECRET</code> is missing / too short	<code>config.env</code> missing or incomplete	copy <code>.env.example</code> <code>config.env</code> and set a secret of ≥ 32 chars
Frontend shows "Cannot reach the server"	backend not running, wrong <code>NEXT_PUBLIC_API_URL</code>	start the backend, check the URL, restart/rebuild the frontend after changing it
Browser console: CORS error	the frontend's origin is not in <code>CORS_ORIGIN</code>	set <code>CORS_ORIGIN=http://localhost:3001</code> (scheme + host + port exactly) in <code>config.env</code> , restart the backend
Login says "Incorrect email or password"	wrong credentials, no admin yet	<code>npm run create-admin</code> ; passwords need ≥ 12 characters
429 <code>RATE_LIMITED</code>	10 failed logins or 300 requests per 15 min per IP	wait <code>Retry-After</code> seconds; raise <code>LOGIN_RATE_MAX</code> / <code>RATE_LIMIT_MAX</code> ; behind a proxy set <code>TRUST_PROXY</code>
401 <code>TOKEN_EXPIRED</code> / user suddenly logged out	token expired (<code>JWT_EXPIRES_IN</code>), account disabled, password changed, <code>JWT_SECRET</code> changed	log in again
403 on import	account is a <code>viewer</code>	use an <code>admin</code> account (the Import button is hidden for viewers)
Upload 400 <code>UPLOAD_MALFORMED</code> / <code>UPLOAD_INVALID</code>	wrong form field	the multipart key must be exactly <code>file</code> (type File) and one file only

Problem	Likely cause	Solution
Upload 415 UNSUPPORTED_FILE_TYPE	not <code>.xlsx</code> / <code>.csv</code>	export the data as <code>.xlsx</code> or <code>.csv</code>
Upload 413 FILE_TOO_LARGE	file over 10 MB	split the file
Upload 422 EXCEL_MISSING_COLUMNS	header row lacks required columns	<code>details.missing</code> lists them; headers must match \$14
409 IMPORT_DUPLICATE_FILE	the exact same file was imported already	revert the old import (history → cancel) if you really want to redo it
409 IMPORT_NOTHING_NEW	every <code>(link, day)</code> already exists	the data is already in the database
Dashboard empty ("No data imported yet")	nothing imported	log in as admin and import a file
Tests: Refusing to run on a database whose name does not contain "test"	<code>DATABASE_TEST</code> wrong	fix it in <code>config.env</code>
Tests skipped	<code>dev-data/congestion_data.xlsx</code> missing	put the workbook there
<code>pnpm</code> fails with <code>ENOSPC</code> : no space left on device	system drive (C:) is full	free space on C: (<code>pnpm</code> 's store and tools live there)
<code>pnpm install</code> fails, version mismatch	<code>packageManager</code> pins <code>pnpm@12.3.4</code>	install that <code>pnpm</code> version (or <code>corepack enable</code>)
Frontend build fails	TypeScript error	run <code>pnpm typecheck</code> and fix the reported file
Strange dependency errors	stale <code>node_modules</code>	delete <code>node_modules</code> and reinstall (<code>npm ci</code> backend; <code>pnpm install --frozen-lockfile</code> frontend)
Docker: <code>docker</code> not recognized / daemon not running	Docker Desktop not installed or not started	install/start Docker Desktop, or use a local MongoDB (§8)
Docker: port 27017 busy	a local <code>mongod</code> is running	stop it, or change the host port in <code>docker-compose.yml</code>

21. DEVELOPMENT WORKFLOW

1. Start MongoDB (§8), then the backend (`npm run dev`), then the frontend (`pnpm dev`).
2. Make the change in the right place:
 - **Business rules / calculations** → `dashboard_ericsson/utils/` (pure functions) or `utils/calc/`. Not in controllers, never in the frontend.
 - **New endpoint** → route (`routes/`) + thin controller (`controllers/`) + logic in `utils/` + a Jest test + an entry in `docs/openapi.yaml` (a test enforces this).
 - **Threshold changes** → `config/thresholds.js` only.
 - **New field on the API** → backend first, then `lib/types.ts`, then the component.

- **Frontend** → components in `components/`, HTTP only through `lib/api.ts`, formatting helpers in `lib/format.ts`.
 - 3. Backend: `npm run jest --runInBand` (everything must pass; no typecheck/lint exists).
 - 4. Frontend: `pnpm typecheck`, then `pnpm build`.
 - 5. Check the diff (`git diff` in the backend repository): no secrets, no `config.env`, no debug code.
 - 6. Try it in the browser, including an error case (wrong password, duplicate upload).
-

22. PRODUCTION BUILD

Backend — no build step. What is in the repository is what runs.

```
cd dashboard_ericsson
npm ci --omit=dev      # install production dependencies only
# set (in config.env or the real environment): NODE_ENV=production, DATABASE_LOCAL, JWT_SECRET,
CORS_ORIGIN=<frontend origin>, TRUST_PROXY=<n> if behind a proxy
npm start              # node server.js
```

The API speaks **plain HTTP**: put a TLS-terminating reverse proxy in front. It must be started from the project folder (it reads `./config.env`). `SIGINT` / `SIGTERM` finish running requests and close the database.

Frontend

```
cd frontend-enhancement-for-dashboard
# .env.local (or the build environment): NEXT_PUBLIC_API_URL=https://<your-api-host>/api/v1
pnpm build                # output folder: .next/
pnpm start                # next start -p 3001
```

`NEXT_PUBLIC_API_URL` is baked in at build time: **rebuild** for each environment. The build produces one statically prerendered page (`/`); the data is fetched in the browser. The backend's `CORS_ORIGIN` must contain the production frontend origin.

Production differences: `NODE_ENV=production`, a strong random `JWT_SECRET`, restricted `CORS_ORIGIN`, MongoDB with authentication and backups, TLS in front of both. No production deployment config (Dockerfile, CI) exists in the project: it needs to be created.

23. SECURITY

- **JWT**: HS256 pinned; secret ≥ 32 characters, validated at startup; checked against the database on every request (disabled user / password change / role change take effect immediately). The token lives in the browser's `localStorage` (readable by any script on the page: keep the frontend free of third-party scripts; no HTTP-only cookie is used).
- **Roles**: `viewer` read-only; `admin` may import and revert. The backend enforces it (`403`); hiding the button in the UI is only cosmetic.
- **Passwords**: bcrypt (`bcryptjs`, cost 12), minimum 12 characters, maximum 72 bytes. Failed-login rate limit: 10 per 15 min per IP.

- **CORS:** only the origins in `CORS_ORIGIN`; methods `GET`, `POST`, `DELETE`, `OPTIONS`; allowed headers `Authorization`, `Content-Type`. Empty value = no CORS headers. `*` must be written explicitly.
- **Helmet** default security headers are on.
- **Rate limiting:** 300 requests / 15 min / IP for the whole API (`/health` excluded). Behind a proxy set `TRUST_PROXY`, or all users share one IP.
- **Input limits:** JSON bodies ≤ 10 kB; one upload of ≤ 10 MB, `.xlsx` / `.csv` only, kept in memory (never written to disk); all query parameters validated before reaching the database (no operator objects, no user-supplied regular expressions).
- **Errors:** no stack traces, MongoDB messages or file paths ever reach the client; they are only in the server log under the same `requestId`. Logs never contain passwords, tokens, the secret or file contents.
- **Secrets:** `config.env`, `.env.local`, real passwords and the `JWT_SECRET` must **never be committed**. `config.env` is git-ignored. Rotate any secret that was shared.
- **MongoDB:** the local container has no authentication and listens on `127.0.0.1` only. For any shared or production server enable authentication, put credentials in `DATABASE_LOCAL`, and do not expose port 27017.
- **Known weakness:** HTTPS is not provided by the app (use a reverse proxy). Tokens cannot be revoked individually before they expire.

24. VERIFIED STATUS AND KNOWN ISSUES

Verification date: **2026-10-07**, Windows 11, Node 22.13.1, npm 10.9.2.

Area	Check	Result
Back end	tests	VERIFIED — <code>npx jest --silent --runInBand</code> : 26/26 suites, 428/428 tests pass
Back end	typecheck	N/A — plain JavaScript, no typecheck exists
Back end	lint	N/A — none configured
Back end	build	N/A — no build step; the server starts and answers <code>/health</code>
Front end	typecheck	VERIFIED — <code>tsc --noEmit</code> exit 0
Front end	build	VERIFIED — <code>next build</code> compiled and generated the static page
Front end	tests / lint	N/A — none configured
Data base	connection	VERIFIED — <code>/health</code> → <code>database: connected</code>
Data base	models and indexes	VERIFIED — read from the live database (\$10)
Integration	login	VERIFIED over HTTP (admin; wrong password → 401; no/bad token → 401)

Area	Check	Result
Integration	dashboard	VERIFIED over HTTP (totals, conditions, top/bottom 10, top-5 wilayas, <code>?date=</code>)
Integration	import	VERIFIED : upload of the real workbook (earlier in this project), duplicate → 409, wrong type → 415, rows/issues endpoints, history. The reference counts in §14 were checked against the stored data. A revert + re-import was done by the project owner (the history shows the original file re-imported later)
Integration	sites / site detail	VERIFIED over HTTP (<code>neId</code> search, filters, 20-bin detail, no-measurement reason)
Integration	wilayas	VERIFIED over HTTP (29 wilayas with sites)
Integration	roles	VERIFIED with a temporary viewer account: viewer <code>GET /sites</code> → 200, <code>POST /imports</code> → 403, <code>DELETE /imports/:id</code> → 403 (the temporary account was deleted afterwards)
Integration	logout	NOT VERIFIED end-to-end — logout only clears <code>localStorage</code> in the browser; there is no endpoint. Not driven in a browser by the author of this report
Frontend UI	login page, logo	VERIFIED by screenshot (headless Edge). The dashboard sidebar and the rest of the UI were not screenshotted by the author of this report (they sit behind the login), and mobile layout is NOT VERIFIED
Docker	<code>docker compose up -d</code>	NOT VERIFIED — Docker is not installed on the machine used
Fresh install	clone → install → run on a clean machine	NOT VERIFIED (see below)

Known issues and caveats (reported, not hidden)

- System drive C: is 100 % full** on the development machine (0 bytes free; D: has ~36 GB). Symptoms seen: `pnpm` could not even start (`ENOSPC`), and the local MongoDB crashed with "out of memory" under parallel test load. The link between the two was not confirmed. Free space before judging anything else.
- pnpm**: `package.json` pins `pnpm@12.3.4`. On this machine `pnpm` could not be launched at the time of verification (point 1), so `pnpm install` was **not re-run**; the frontend commands in this report that were run (typecheck, build, start) used `npx` against the existing `node_modules`. `pnpm dev` / `pnpm build` / `pnpm start` are the scripts defined in `package.json` but were not executed through `pnpm`. `npm install` as an alternative is untested.
- Parallel tests crash the local MongoDB** on this machine; use `--runInBand` (§19).
- The thresholds 50 / 80 are provisional** and need engineering confirmation (`config/thresholds.js`).
- No preview/commit step, no `theta` parameter, no logout endpoint, no API-docs web page** exist (§12–§15). Some project briefs assumed they do.
- Source control**: the backend repository has many uncommitted modifications and new files (`README.md` , `docker-compose.yml` , `docs/` , several tests...); the frontend has no Git repository; the project root is not a repository. Decide the repository layout and commit before handing over.
- `dev-data/congestion_data.xlsx` is git-ignored**: provide it separately.

8. Accounts present in the development database: `admin@example.com` (created by this work, with a development password that was shared in the working session: **change or delete it**), `admin@demo.local`, `viewer@demo.local` (already present; not created by this work). Passwords are not recorded here.
9. Folders named `.npm-cache/` exist inside both projects (local npm caches); the backend ignores it in Git, the frontend `.gitignore` was extended to ignore it too.
10. A second, older copy of the frontend exists at `D:\samer\Documents\frontend-enhancement-for-dashboard\` (not integrated; also contains the original logo files). Delete or archive it to avoid confusion.

Is the project runnable from a fresh setup? Everything needed is in the repository except: the real workbook (git-ignored), a created `config.env`, and a MongoDB. By the steps in §1 it **should** run, and each step was executed on the development machine individually. A truly fresh-machine run (clone → install → run) **was not performed**, and Docker was not available to test the Docker route. Treat the first run on a clean machine as the final acceptance test (use the checklist below).

25. HANDOVER CHECKLIST

```
[ ] Node.js 22 installed                (node -v)
[ ] pnpm 12 installed                  (pnpm -v)
[ ] Enough free disk space (>1 GB on the system drive)
[ ] Dependencies installed              (npm install / pnpm install)
[ ] config.env created, JWT_SECRET (>=32 chars) set, CORS_ORIGIN=http://localhost:3001
[ ] .env.local created in the frontend (optional: default works)
[ ] MongoDB running on 127.0.0.1:27017  (docker compose ps / /health shows "connected")
[ ] First admin created                 (npm run create-admin; ADMIN_PASSWORD then removed)
[ ] Backend running                    (http://localhost:3000/api/v1/health)
[ ] Frontend running                   (http://localhost:3001)
[ ] Login working                      (admin account)
[ ] Import working                     (upload congestion_data.xlsx: 1,200 rows / 862 valid / 338
failed)
[ ] Dashboard working                  (1,194 sites for that file alone; 151 measured on 2026-02-
10)
[ ] Sites working                      (search NeId 18870 → 62.7 % medium)
[ ] Site details working               (20-slice histogram; no-measurement reason on a grey site)
[ ] Wilayas working                   (filter dropdown + top-5 on the dashboard)
[ ] Tests passing                      (npx jest --runInBand → 428 passed)
[ ] Frontend typecheck + build passing (pnpm typecheck, pnpm build)
[ ] Secrets not committed              (config.env, .env.local, passwords)
[ ] Repository layout decided and code committed
```